Build System

[??]

This document explains the implementation of the ESP-IDF build system and the concept of "components". Read this document if you want to know how to organize and build a new ESP-IDF project or component.

Overview

An ESP-IDF project can be seen as an amalgamation of a number of components. For example, for a web server that shows the current humidity, there could be:

- The ESP-IDF base libraries (libc, ROM bindings, etc)
- The Wi-Fi drivers
- A TCP/IP stack
- The FreeRTOS operating system
- A web server
- A driver for the humidity sensor
- Main code tying it all together

ESP-IDF makes these components explicit and configurable. To do that, when a project is compiled, the build system will look up all the components in the ESP-IDF directories, the project directories and (optionally) in additional custom component directories. It then allows the user to configure the ESP-IDF project using a text-based menu system to customize each component. After the components in the project are configured, the build system will compile the project.

Concepts

- A `project` is a directory that contains all the files and configuration to build a single `app` (executable), as well as additional supporting elements such as a partition table, data partitions or filesystem partitions, and a bootloader.
- `Project configuration` is held in a single file called `sdkconfig` in the root directory of the project. This configuration file is modified via `idf.py menuconfig` to customize the configuration of the project. A single project contains exactly one project configuration.
- An `app` is an executable that is built by ESP-IDF. A single project will usually build two apps - a "project app" (the main executable, ie your custom firmware) and a "bootloader app" (the initial bootloader program which launches the project app).

- `Components` are modular pieces of standalone code that are compiled into static libraries (.a files) and linked to an app. Some are provided by ESP-IDF itself, others may be sourced from other places.
- `Target` is the hardware for which an application is built. A full list of supported targets in your version of ESP-IDF can be seen by running *idf.py --list-targets*.

Some things are not part of the project:

- `ESP-IDF` is not part of the project. Instead, it is standalone, and linked to the project via the `IDF_PATH` environment variable which holds the path of the `esp-idf` directory. This allows the ESP-IDF framework to be decoupled from your project.
- The toolchain for compilation is not part of the project. The toolchain should be installed in the system command line PATH.

Using the Build System

idf.py

The `idf.py` command-line tool provides a front-end for easily managing your project builds. It manages the following tools:

- CMake, which configures the project to be built
- Ninja which builds the project
- esptool for flashing the target.

For more details about configuring the build system using `idf.py`, please refer to IDF Frontend.

Using CMake Directly

idf.py is a wrapper around CMake for convenience. However, you can also invoke CMake directly.

When `idf.py` does something, it prints each command that it runs for easy reference. For example, the `idf.py build` command is the same as running these commands in a bash shell (or similar commands for Windows Command Prompt):

```
mkdir -p build
cd build
cmake .. -G Ninja   # or 'Unix Makefiles'
ninja
```

In the above list, the `cmake` command configures the project and generates build files for use with the final build tool. In this case, the final build tool is Ninja: running `ninja` actually builds the project.

It's not necessary to run `cmake` more than once. After the first build, you only need to run `ninja` each time. `ninja` will automatically re-invoke `cmake` if the project needs reconfiguration.

If using CMake with `ninja` or `make`, there are also targets for more of the `idf.py` sub-commands. For example, running `make menuconfig` or `ninja menuconfig` in the build directory will work the same as `idf.py menuconfig`.

Note

If you're already familiar with CMake, you may find the ESP-IDF CMake-based build system unusual because it wraps a lot of CMake's functionality to reduce boilerplate. See writing pure CMake components for some information about writing more "CMake style" components.

Flashing with Ninja or Make

It's possible to build and flash directly from ninja or make by running a target like:

```
ninja flash
```

Or:

```
make app-flash
```

Available targets are: `flash`, `app-flash` (app only), `bootloader-flash` (bootloader only).

When flashing this way, optionally set the `ESPPORT` and `ESPBAUD` environment variables to specify the serial port and baud rate. You can set environment variables in your operating system or IDE project. Alternatively, set them directly on the command line:

```
ESPPORT=/dev/ttyUSB0 ninja flash
```

Note

Providing environment variables at the start of the command like this is Bash shell Syntax. It will work on Linux and macOS. It won't work when using Windows Command Prompt, but it will work when using Bash-like shells on Windows.

Or:

```
make -j3 app-flash ESPPORT=COM4 ESPBAUD=2000000
```

Note

Providing variables at the end of the command line is `make` syntax, and works for `make` on all platforms.

Using CMake in an IDE

You can also use an IDE with CMake integration. The IDE will want to know the path to the project's `CMakeLists.txt` file. IDEs with CMake integration often provide their own build tools (CMake calls these "generators") to build the source files as part of the IDE.

When adding custom non-build steps like "flash" to the IDE, it is recommended to execute `idf.py` for these "special" commands.

For more detailed information about integrating ESP-IDF with CMake into an IDE, see Build System Metadata.

Setting up the Python Interpreter

ESP-IDF works well with Python version 3.10+.

`idf.py` and other Python scripts will run with the default Python interpreter, i.e., `python`. You can switch to a different one like `python3 $IDF_PATH/tools/idf.py ...`, or you can set up a shell alias or another script to simplify the command.

If using CMake directly, running `cmake -D PYTHON=python3 ...` will cause CMake to override the default Python interpreter.

If using an IDE with CMake, setting the `PYTHON` value as a CMake cache override in the IDE UI will override the default Python interpreter.

To manage the Python version more generally via the command line, check out the tools pyenv or virtualenv. These let you change the default Python version.

Example Project

An example project directory tree might look like this:

```
- myProject/
             - CMakeLists.txt
             - sdkconfig
             - dependencies.lock
             - bootloader_components/ - boot_component/ - CMakeLists.txt
                                                        - Kconfig
                                                        - src1.c
             - components/ - component1/ - CMakeLists.txt
                                         - Kconfig
                                         - src1.c
                           - component2/ - CMakeLists.txt
                                         - Kconfig
                                         - src1.c
                                         - include/ - component2.h
             - managed_components/ - namespace__component-name/ - CMakelists.txt
                                                                - src1.c
                                                                - idf_component.yml
                                                                - include/ - src1.h
             - main/       - CMakeLists.txt
                           - src1.c
                           - src2.c
                           - idf_component.yml
             - build/
```

This example "myProject" contains the following elements:

- A top-level project CMakeLists.txt file. This is the primary file which CMake uses to learn how to build the project; and may set project-wide CMake variables. It includes the file /tools/cmake/project.cmake which implements the rest of the build system. Finally, it sets the project name and defines the project.
- "sdkconfig" project configuration file. This file is created/updated when `idf.py menuconfig` runs, and holds the configuration for all of the components in the project (including ESP-IDF itself). The `sdkconfig` file may or may not be added to the source control system of the project. More information about this file can be found in the sdkconfig file section in the Configuration Guide.
- "dependencies.lock" file contains the list of all managed components, and their versions, that are currently in used in the project. The `dependencies.lock` file is generated or updated automatically when IDF Component Manager is used to add or update project components. So this file should never be edited manually! If the project does not have `idf_component.yml` files in any of its components, `dependencies.lock` will not be created.
- Optional "idf_component.yml" file contains metadata about the component and its dependencies. It is used by the IDF Component Manager to download and resolve these dependencies. More information about this file can be found in the idf_component.yml section.

- Optional "bootloader_components" directory contains components that need to be compiled and linked inside the bootloader project. A project does not have to contain custom bootloader components of this kind, but it can be useful in case the bootloader needs to be modified to embed new features.
- Optional "components" directory contains components that are part of the project. A project does not have to contain custom components of this kind, but it can be useful for structuring reusable code or including third-party components that aren't part of ESP-IDF. Alternatively, `EXTRA_COMPONENT_DIRS` can be set in the top-level CMakeLists.txt to look for components in other places.
- "main" directory is a special component that contains source code for the project itself. "main" is a default name, the CMake variable `COMPONENT_DIRS` includes this component but you can modify this variable. See the renaming main section for more info. If you have a lot of source files in your project, we recommend grouping most into components instead of putting them all in "main".
- "build" directory is where the build output is created. This directory is created by `idf.py` if it doesn't already exist. CMake configures the project and generates interim build files in this directory. Then, after the main build process is run, this directory will also contain interim object files and libraries as well as final binary output files. This directory is usually not added to source control or distributed with the project source code.
- "managed_components" directory is created by the IDF Component Manager to store components managed by this tool. Each managed component typically includes a `idf_component.yml` manifest file defining the component's metadata, such as version and dependencies. However, for components sourced from Git repositories, the manifest file is optional. Users should avoid manually modifying the contents of the "managed_components" directory. If alterations are needed, the component can be copied to the `components` directory. The "managed_components" directory is usually not versioned in Git and not distributed with the project source code.

Component directories each contain a component `CMakeLists.txt` file. This file contains variable definitions to control the build process of the component, and its integration into the overall project. See Component CMakeLists Files for more details.

Each component may also include a `Kconfig` file defining the component configuration options that can be set via `menuconfig`. Some components may also include `Kconfig.projbuild` and `project_include.cmake` files, which are special files for overriding parts of the project.

Project CMakeLists File

Each project has a single top-level `CMakeLists.txt` file that contains build settings for the entire project. By default, the project CMakeLists can be quite minimal.

Minimal Example CMakeLists

Minimal project:

```
cmake_minimum_required(VERSION 3.22)
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(myProject)
```

Mandatory Parts

The inclusion of these three lines, in the order shown above, is necessary for every project:

- `cmake_minimum_required(VERSION 3.22)` tells CMake the minimum version that is required to build the project. ESP-IDF is designed to work with CMake 3.22 or newer. This line must be the first line in the CMakeLists.txt file.
- `include($ENV{IDF_PATH}/tools/cmake/project.cmake)` pulls in the rest of the CMake functionality to configure the project, discover all the components, etc.
- `project(myProject)` creates the project itself, and specifies the project name. The project name is used for the final binary output files of the app - ie `myProject.elf`, `myProject.bin`. Only one project can be defined per CMakeLists file.

Optional Project Variables

These variables all have default values that can be overridden for custom behavior. Look in /tools/cmake/project.cmake for all of the implementation details.

- `COMPONENT_DIRS`: Directories to search for components. Defaults to `IDF_PATH/components`, `PROJECT_DIR/components`, and `EXTRA_COMPONENT_DIRS`. Override this variable if you don't want to search for components in these places.
- `EXTRA_COMPONENT_DIRS`: Optional list of additional directories to search for components. Paths can be relative to the project directory, or absolute.
- `COMPONENTS`: A list of component names to build into the project. Defaults to all components found in the `COMPONENT_DIRS` directories. Use this variable to "trim down" the project for faster build times. Note that any component which "requires" another component via the REQUIRES or PRIV_REQUIRES arguments on component registration will automatically have it added to this list, so the `COMPONENTS` list can be very short. The `MINIMAL_BUILD` build property can be used as an alternative to specifying only the `main` component in `COMPONENTS`.
- `BOOTLOADER_IGNORE_EXTRA_COMPONENT`: Optional list of components, placed in `bootloader_components/`, that should be ignored by the bootloader compilation. Use this variable if a bootloader component needs to be included conditionally inside the project.

- `BOOTLOADER_EXTRA_COMPONENT_DIRS`: Optional list of additional directories to search for components to be compiled as part of the bootloader. Please note that this is a build property.

Any paths in these variables can be absolute paths, or set relative to the project directory.

To set these variables, use the cmake set command ie `set(VARIABLE "VALUE")`. The `set()` commands should be placed after the `cmake_minimum(...)` line but before the `include(...)` line.

Renaming `main` Component

The build system provides special treatment to the `main` component. It is a component that gets automatically added to the build provided that it is in the expected location, PROJECT_DIR/main. All other components in the build are also added as its dependencies, saving the user from hunting down dependencies and providing a build that works right out of the box. Renaming the `main` component causes the loss of these behind-the-scenes heavy lifting, requiring the user to specify the location of the newly renamed component and manually specify its dependencies. Specifically, the steps to renaming `main` are as follows:

1. Rename `main` directory.
2. Set `EXTRA_COMPONENT_DIRS` in the project CMakeLists.txt to include the renamed `main` directory.
3. Specify the dependencies in the renamed component's CMakeLists.txt file via REQUIRES or PRIV_REQUIRES arguments on component registration.

> **Note**
>
> A project without a `main` component cannot use the `MINIMAL_BUILD` build property, as this property explicitly relies on the presence of the `main` component. Ensure that the `MINIMAL_BUILD` build property is not set for projects that do not include a `main` component.

Overriding Default Build Specifications

The build sets some global build specifications (compile flags, definitions, etc.) that gets used in compiling all sources from all components.

For example, one of the default build specifications set is the compile option `-Wextra`. Suppose a user wants to use override this with `-Wno-extra`, it should be done after `project()`:

```
cmake_minimum_required(VERSION 3.22)
include($ENV{IDF_PATH}/tools/cmake/project.cmake)
project(myProject)

idf_build_set_property(COMPILE_OPTIONS "-Wno-error" APPEND)
```

This ensures that the compile options set by the user won't be overridden by the default build specifications, since the latter are set inside `project()`.

Component CMakeLists Files

Each project contains one or more components. Components can be part of ESP-IDF, part of the project's own components directory, or added from custom component directories (see above).

A component is any directory in the `COMPONENT_DIRS` list which contains a `CMakeLists.txt` file.

Searching for Components

The list of directories in `COMPONENT_DIRS` is searched for the project's components. Directories in this list can either be components themselves (ie they contain a *CMakeLists.txt* file), or they can be top-level directories whose sub-directories are components.

When CMake runs to configure the project, it logs the components included in the build. This list can be useful for debugging the inclusion/exclusion of certain components.

Multiple Components with the Same Name

When ESP-IDF is collecting all the components to compile, the search precedence is as follows (from highest to lowest):

- Project components
- Components from `EXTRA_COMPONENT_DIRS`
- Project managed components, downloaded by the IDF Component Manager into `PROJECT_DIR/managed_components`, unless the IDF Component Manager is disabled.
- ESP-IDF components (`IDF_PATH/components`)

If two or more of these directories contain component sub-directories with the same name, the component with higher precedence is used. This allows, for example, overriding ESP-IDF components with a modified version by copying that component from the ESP-IDF components directory to the project components directory and then modifying it there. If used in this way, the ESP-IDF directory itself can remain untouched.

Note

If a component is overridden in an existing project by moving it to a new location, the project will not automatically see the new component path. Run `idf.py reconfigure` (or delete the project build folder) and then build again.

Minimal Component CMakeLists

The minimal component `CMakeLists.txt` file simply registers the component to the build system using `idf_component_register`:

```
idf_component_register(SRCS "foo.c" "bar.c"
                       INCLUDE_DIRS "include"
                       REQUIRES mbedtls)
```

- `SRCS` is a list of source files (`*.c`, `*.cpp`, `*.cc`, `*.S`). These source files will be compiled into the component library.
- `INCLUDE_DIRS` is a list of directories to add to the global include search path for any component which requires this component, and also the main source files.
- `REQUIRES` is not actually required, but it is very often required to declare what other components this component will use. See component requirements.

A library with the name of the component will be built and linked to the final app.

Directories are usually specified relative to the `CMakeLists.txt` file itself, although they can be absolute.

There are other arguments that can be passed to `idf_component_register`. These arguments are discussed here.

See example component requirements and example component CMakeLists for more complete component `CMakeLists.txt` examples.

Preset Component Variables

The following component-specific variables are available for use inside component CMakeLists, but should not be modified:

- `COMPONENT_DIR`: The component directory. Evaluates to the absolute path of the directory containing `CMakeLists.txt`. The component path cannot contain spaces. This is the same as the `CMAKE_CURRENT_SOURCE_DIR` variable.
- `COMPONENT_NAME`: Name of the component. Same as the name of the component directory.

- `COMPONENT_ALIAS`: Alias of the library created internally by the build system for the component.
- `COMPONENT_LIB`: Name of the library created internally by the build system for the component.
- `COMPONENT_VERSION`: Component version specified by idf_component.yml and set by IDF Component Manager.

The following variables are set at the project level, but available for use in component CMakeLists:

- `CONFIG_*`: Each value in the project configuration has a corresponding variable available in cmake. All names begin with `CONFIG_`. More information on how the project configuration works, please visit Project Configuration Guide.
- `ESP_PLATFORM`: Set to 1 when the CMake file is processed within the ESP-IDF build system.

Build/Project Variables

The following are some project/build variables that are available as build properties and whose values can be queried using `idf_build_get_property` from the component CMakeLists.txt:

- `PROJECT_NAME`: Name of the project, as set in project CMakeLists.txt file.
- `PROJECT_DIR`: Absolute path of the project directory containing the project CMakeLists. Same as the `CMAKE_SOURCE_DIR` variable.
- `COMPONENTS`: Names of all components that are included in this build, formatted as a semicolon-delimited CMake list.
- `IDF_VER`: Git version of ESP-IDF (produced by `git describe`)
- `IDF_VERSION_MAJOR`, `IDF_VERSION_MINOR`, `IDF_VERSION_PATCH`: Components of ESP-IDF version, to be used in conditional expressions. Note that this information is less precise than that provided by `IDF_VER` variable. `v4.0-dev-*`, `v4.0-beta1`, `v4.0-rc1` and `v4.0` will all have the same values of `IDF_VERSION_*` variables, but different `IDF_VER` values.
- `IDF_TARGET`: Name of the target for which the project is being built.
- `PROJECT_VER`: Project version.
  - If CONFIG_APP_PROJECT_VER_FROM_CONFIG option is set, the value of CONFIG_APP_PROJECT_VER will be used.
  - Else, if `PROJECT_VER` variable is set in project CMakeLists.txt file, its value will be used.
  - Else, if the `PROJECT_DIR/version.txt` exists, its contents will be used as `PROJECT_VER`.
  - Else, if `VERSION` argument is passed to the `project()` call in the CMakeLists.txt file as `project(... VERSION x.y.z.w )` then it will be used as `PROJECT_VER`. The `VERSION` argument must be compliant with the cmake standard.
  - Else, if the project is located inside a Git repository, the output of git description will be used.

    - Otherwise, `PROJECT_VER` will be "1".
- `EXTRA_PARTITION_SUBTYPES`: CMake list of extra partition subtypes. Each subtype description is a comma-separated string with `type_name, subtype_name, numeric_value` format. Components may add new subtypes by appending them to this list.

Other build properties are listed here.

Controlling Component Compilation

To pass compiler options when compiling source files belonging to a particular component, use the target_compile_options function:

```
target_compile_options(${COMPONENT_LIB} PRIVATE -Wno-unused-variable)
```

To apply the compilation flags to a single source file, use the CMake set_source_files_properties command:

```
set_source_files_properties(mysrc.c
    PROPERTIES COMPILE_FLAGS
    -Wno-unused-variable
)
```

This can be useful if there is upstream code that emits warnings.

> **Note**
>
> CMake set_source_files_properties command is not applicable when the source files have been populated with help of the `SRC_DIRS` variable in `idf_component_register`. See File Globbing & Incremental Builds for more details.

When using these commands, place them after the call to `idf_component_register` in the component CMakeLists file.

Component Configuration

Each component can also have a `Kconfig` file, alongside `CMakeLists.txt`. This contains configuration settings to add to the configuration menu for this component.

These settings are found under the "Component Settings" menu when menuconfig is run.

To create a component Kconfig file, it is easiest to start with one of the Kconfig files distributed with ESP-IDF.

For an example, see Adding conditional configuration. For a more detailed guide, see Component Configuration Guide.

Preprocessor Definitions

The ESP-IDF build system adds the following C preprocessor definitions on the command line:

- `ESP_PLATFORM` : Can be used to detect that build happens within ESP-IDF.
- `IDF_VER` : Defined to a git version string. E.g. `v2.0` for a tagged release or `v1.0-275-g0efaa4f` for an arbitrary commit.

Component Requirements

When compiling each component, the ESP-IDF build system recursively evaluates its dependencies. This means each component needs to declare the components that it depends on ("requires").

When Writing a Component

```
idf_component_register(...
                       REQUIRES mbedtls
                       PRIV_REQUIRES console spiffs)
```

- `REQUIRES` should be set to all components whose header files are #included from the *public* header files of this component.
- `PRIV_REQUIRES` should be set to all components whose header files are #included from *any source files* in this component, unless already listed in `REQUIRES`. Also, any component which is required to be linked in order for this component to function correctly.
- The values of `REQUIRES` and `PRIV_REQUIRES` should not depend on any configuration options (`CONFIG_xxx` macros). This is because requirements are expanded before the configuration is loaded. Other component variables (like include paths or source files) can depend on configuration options.
- Not setting either or both `REQUIRES` variables is fine. If the component has no requirements except for the Common component requirements needed for RTOS, libc, etc.

If a component only supports some target chips (values of `IDF_TARGET`) then it can specify `REQUIRED_IDF_TARGETS` in the `idf_component_register` call to express these requirements. In this case, the build system will generate an error if the component is included in the build, but does not support the selected target.

> **Note**
>
> In CMake terms, `REQUIRES` & `PRIV_REQUIRES` are approximate wrappers around the CMake functions `target_link_libraries(... PUBLIC ...)` and `target_link_libraries(... PRIVATE ...)`.

Example of Component Requirements

Imagine there is a `car` component, which uses the `engine` component, which uses the `spark_plug` component:

```
- autoProject/
             - CMakeLists.txt
             - components/ - car/ - CMakeLists.txt
                                  - car.c
                                  - car.h
                           - engine/ - CMakeLists.txt
                                     - engine.c
                                     - include/ - engine.h
                           - spark_plug/  - CMakeLists.txt
                                          - spark_plug.c
                                          - spark_plug.h
```

Car Component

The `car.h` header file is the public interface for the `car` component. This header includes `engine.h` directly because it uses some declarations from this header:

```
/* car.h */
#include "engine.h"

#ifdef ENGINE_IS_HYBRID
#define CAR_MODEL "Hybrid"
#endif
```

And car.c includes `car.h` as well:

```
/* car.c */
#include "car.h"
```

This means the `car/CMakeLists.txt` file needs to declare that `car` requires `engine`:

```
idf_component_register(SRCS "car.c"
                       INCLUDE_DIRS "."
                       REQUIRES engine)
```

- `SRCS` gives the list of source files in the `car` component.
- `INCLUDE_DIRS` gives the list of public include directories for this component. Because the public interface is `car.h`, the directory containing `car.h` is listed here.
- `REQUIRES` gives the list of components required by the public interface of this component. Because `car.h` is a public header and includes a header from `engine`, we include `engine` here. This makes sure that any other component which includes `car.h` will be able to recursively include the required `engine.h` also.

Engine Component

The `engine` component also has a public header file `include/engine.h`, but this header is simpler:

```
/* engine.h */
#define ENGINE_IS_HYBRID

void engine_start(void);
```

The implementation is in `engine.c`:

```
/* engine.c */
#include "engine.h"
#include "spark_plug.h"

...
```

In this component, `engine` depends on `spark_plug` but this is a private dependency. `spark_plug.h` is needed to compile `engine.c`, but not needed to include `engine.h`.

This means that the `engine/CMakeLists.txt` file can use `PRIV_REQUIRES`:

```
idf_component_register(SRCS "engine.c"
                       INCLUDE_DIRS "include"
                       PRIV_REQUIRES spark_plug)
```

As a result, source files in the `car` component don't need the `spark_plug` include directories added to their compiler search path. This can speed up compilation, and stops compiler command lines from becoming longer than necessary.

Spark Plug Component

The `spark_plug` component doesn't depend on anything else. It has a public header file `spark_plug.h`, but this doesn't include headers from any other components.

This means that the `spark_plug/CMakeLists.txt` file doesn't need any `REQUIRES` or `PRIV_REQUIRES` clauses:

```
idf_component_register(SRCS "spark_plug.c"
                       INCLUDE_DIRS ".")
```

Source File Include Directories

Each component's source file is compiled with these include path directories, as specified in the passed arguments to `idf_component_register`:

```
idf_component_register(..
                       INCLUDE_DIRS "include"
                       PRIV_INCLUDE_DIRS "other")
```

- The current component's `INCLUDE_DIRS` and `PRIV_INCLUDE_DIRS`.
- The `INCLUDE_DIRS` belonging to all other components listed in the `REQUIRES` and `PRIV_REQUIRES` parameters (ie all the current component's public and private dependencies).
- Recursively, all of the `INCLUDE_DIRS` of those components `REQUIRES` lists (ie all public dependencies of this component's dependencies, recursively expanded).

Main Component Requirements

The component named `main` is special because it automatically requires all other components in the build. So it's not necessary to pass `REQUIRES` or `PRIV_REQUIRES` to this component. See renaming main for a description of what needs to be changed if no longer using the `main` component.

Common Component Requirements

To avoid duplication, every component automatically requires some "common" IDF components even if they are not mentioned explicitly. Headers from these components can always be included.

The list of common components is: cxx, esp_libc, freertos, esp_hw_support, heap, log, soc, hal, esp_rom, esp_common, esp_system, xtensa/riscv.

Including Components in the Build

- By default, every component is included in the build.
- If you set the `COMPONENTS` variable to a minimal list of components used directly by your project, then the build will expand to also include required components. The full list of components will be:
  - Components mentioned explicitly in `COMPONENTS`.
  - Those components' requirements (evaluated recursively).
  - The common components that every component depends on.
- Setting `COMPONENTS` to the minimal list of required components can significantly reduce compile times.
- The `MINIMAL_BUILD` build property can be set to `ON`, which acts as a shortcut to configure the `COMPONENTS` variable to include only the `main` component. This means that the build will include only the common components, the `main` component, and all dependencies associated with it, both direct and indirect. If the `COMPONENTS` variable is defined while the `MINIMAL_BUILD` property is enabled, `COMPONENTS` will take precedence.

> **Note**
>
> Certain features and configurations, such as those provided by esp_psram or espcoredump components, may not be available to your project by default if the minimal list of components is used. When using the `COMPONENTS` variable, ensure that all necessary components are included. Similarly, when using the `MINIMAL_BUILD` build property, ensure that all required components are specified in the `REQUIRES` or `PRIV_REQUIRES` argument during component registration.

Circular Dependencies

It's possible for a project to contain Component A that requires (`REQUIRES` or `PRIV_REQUIRES`) Component B, and Component B that requires Component A. This is known as a dependency cycle or a circular dependency.

CMake will usually handle circular dependencies automatically by repeating the component library names twice on the linker command line. However this strategy doesn't always work, and the build may fail with a linker error about "Undefined reference to ...", referencing a symbol defined by one of the components inside the circular dependency. This is particularly likely if there is a large circular dependency, i.e., A > B > C > D > A.

The best solution is to restructure the components to remove the circular dependency. In most cases, a software architecture without circular dependencies has desirable properties of modularity and clean layering and will be more maintainable in the long term. However, removing circular dependencies is not always possible.

To bypass a linker error caused by a circular dependency, the simplest workaround is to increase the CMake LINK_INTERFACE_MULTIPLICITY property of one of the component libraries. This causes CMake to repeat this library and its dependencies more than two times on the linker command line.

For example:

```
set_property(TARGET ${COMPONENT_LIB} APPEND PROPERTY LINK_INTERFACE_MULTIPLICITY 3)
```

- This line should be placed after `idf_component_register` in the component CMakeLists.txt file.
- If possible, place this line in the component that creates the circular dependency by depending on a lot of other components. However, the line can be placed inside any component that is part of the cycle. Choosing the component that owns the source file shown in the linker error message, or the component that defines the symbol(s) mentioned in the linker error message, is a good place to start.
- Usually increasing the value to 3 (default is 2) is enough, but if this doesn't work then try increasing the number further.
- Adding this option will make the linker command line longer, and the linking stage slower.

Advanced Workaround: Undefined Symbols

If only one or two symbols are causing a circular dependency, and all other dependencies are linear, then there is an alternative method to avoid linker errors: Specify the specific symbols required for the "reverse" dependency as undefined symbols at link time.

For example, if component A depends on component B but component B also needs to reference `reverse_ops` from component A (but nothing else), then you can add a line like the following to the component B CMakeLists.txt to resolve the cycle at link time:

```
# This symbol is provided by 'Component A' at link time
target_link_libraries(${COMPONENT_LIB} INTERFACE "-u reverse_ops")
```

- The `-u` argument means that the linker will always include this symbol in the link, regardless of dependency ordering.
- This line should be placed after `idf_component_register` in the component CMakeLists.txt file.
- If 'Component B' doesn't need to access any headers of 'Component A', only link to a few symbol(s), then this line can be used instead of any `REQUIRES` from B to A. This further simplifies the component structure in the build system.

See the target_link_libraries documentation for more information about this CMake function.

Requirements in the Build System Implementation

- Very early in the CMake configuration process, the script `expand_requirements.cmake` is run. This script does a partial evaluation of all component CMakeLists.txt files and builds a graph of component requirements (this graph may have cycles). The graph is used to generate a file `component_depends.cmake` in the build directory.
- The main CMake process then includes this file and uses it to determine the list of components to include in the build (internal `BUILD_COMPONENTS` variable). The `BUILD_COMPONENTS` variable is sorted so dependencies are listed first, however, as the component dependency graph has cycles this cannot be guaranteed for all components. The order should be deterministic given the same set of components and component dependencies.
- The value of `BUILD_COMPONENTS` is logged by CMake as "Component names: "
- Configuration is then evaluated for the components included in the build.
- Each component is included in the build normally and the CMakeLists.txt file is evaluated again to add the component libraries to the build.

Component Dependency Order

The order of components in the `BUILD_COMPONENTS` variable determines other orderings during the build:

- Order that Project_include.cmake files are included in the project.
- Order that the list of header paths is generated for compilation (via `-I` argument). (Note that for a given component's source files, only that component's dependency's header paths are passed to the compiler.)

Adding Link-Time Dependencies

The ESP-IDF CMake helper function `idf_component_add_link_dependency` adds a link-only dependency between one component and another. In almost all cases, it is better to use the `PRIV_REQUIRES` feature in `idf_component_register` to create a dependency. However, in some cases, it's necessary to add the link-time dependency of another component to this component, i.e., the reverse order to `PRIV_REQUIRES` (for example: Overriding Default Chip Drivers).

To make another component depend on this component at link time:

```
idf_component_add_link_dependency(FROM other_component)
```

Place this line after the line with `idf_component_register`.

It's also possible to specify both components by name:

```
idf_component_add_link_dependency(FROM other_component TO that_component)
```

Overriding Parts of the Project

Project_include.cmake

For components that have build requirements that must be evaluated before any component CMakeLists files are evaluated, you can create a file called `project_include.cmake` in the component directory. This CMake file is included when `project.cmake` is evaluating the entire project.

`project_include.cmake` files are used inside ESP-IDF, for defining project-wide build features such as `esptool` command line arguments and the `bootloader` "special app".

Unlike component `CMakeLists.txt` files, when including a `project_include.cmake` file the current source directory (`CMAKE_CURRENT_SOURCE_DIR` and working directory) is the project directory. Use the variable `COMPONENT_DIR` for the absolute directory of the component.

Note that `project_include.cmake` isn't necessary for the most common component uses, such as adding include directories to the project, or `LDFLAGS` to the final linking step. These values can be customized via the `CMakeLists.txt` file itself. See Optional Project Variables for details.

`project_include.cmake` files are included in the order given in `BUILD_COMPONENTS` variable (as logged by CMake). This means that a component's `project_include.cmake` file will be included after it's all dependencies' `project_include.cmake` files, unless both components are part of a dependency cycle. This is important if a `project_include.cmake` file relies on variables set by another component. See also above.

Take great care when setting variables or targets in a `project_include.cmake` file. As the values are included in the top-level project CMake pass, they can influence or break functionality across all components!

Kconfig.projbuild

This is an equivalent to `project_include.cmake` for Component Configuration Kconfig files. If you want to include configuration options at the top level of menuconfig, rather than inside the "Component Configuration" sub-menu, then these can be defined in the Kconfig.projbuild file alongside the `CMakeLists.txt` file.

Take care when adding configuration values in this file, as they will be included across the entire project configuration. Where possible, it's generally better to create a Kconfig file for Component Configuration.

For more information, see Kconfig Files section in the Configuration Guide.

Wrappers to Redefine or Extend Existing Functions

Thanks to the linker's wrap feature, it is possible to redefine or extend the behavior of an existing ESP-IDF function. To do so, you will need to provide the following CMake declaration in your project's `CMakeLists.txt` file:

```
target_link_libraries(${COMPONENT_LIB} INTERFACE "-Wl,--wrap=function_to_redefine")
```

Where `function_to_redefine` is the name of the function to redefine or extend. This option will let the linker replace all the calls to `function_to_redefine` functions in the binary libraries with calls to `__wrap_function_to_redefine` function. Thus, you must define this new symbol in your application.

The linker will provide a new symbol named `__real_function_to_redefine` which points to the former implementation of the function to redefine. It can be called from the new implementation, making it an extension of the former one.

This mechanism is shown in the example build_system/wrappers. Check examples/build_system/wrappers/README.md for more details.

Override the Default Bootloader

Thanks to the optional `bootloader_components` directory present in your ESP-IDF project, it is possible to override the default ESP-IDF bootloader. To do so, a new `bootloader_components/main` component should be defined, which will make the project directory tree look like the following:

- **myProject/**
    - CMakeLists.txt
    - sdkconfig
    - **bootloader_components/ - main/ - CMakeLists.txt**
        - Kconfig
        - my_bootloader.c
    - **main/ - CMakeLists.txt**
        - app_main.c
    - build/

Here, the `my_bootloader.c` file becomes source code for the new bootloader, which means that it will need to perform all the required operations to set up and load the `main` application from flash.

It is also possible to conditionally replace the bootloader depending on a certain condition, such as the target for example. This can be achieved thanks to the `BOOTLOADER_IGNORE_EXTRA_COMPONENT` CMake variable. This list can be used to tell the ESP-IDF bootloader project to ignore and not compile the given components present in `bootloader_components`. For example, if one wants to use the default bootloader for ESP32 target, then `myProject/CMakeLists.txt` should look like the following:

```
include($ENV{IDF_PATH}/tools/cmake/project.cmake)

if(${IDF_TARGET} STREQUAL "esp32")
    set(BOOTLOADER_IGNORE_EXTRA_COMPONENT "main")
endif()

project(main)
```

It is important to note that this can also be used for any other bootloader components than `main`. In all cases, the prefix `bootloader_component` must not be specified.

See custom_bootloader/bootloader_override for an example of overriding the default bootloader.

Similarly to regular applications, it is possible to include external components, not placed in *bootloader_component*, as part of the bootloader build thanks to the build property `BOOTLOADER_EXTRA_COMPONENT_DIRS`. It can either refer to a directory that contains several components, or refer to a single component. For example:

```
include($ENV{IDF_PATH}/tools/cmake/project.cmake)

idf_build_set_property(BOOTLOADER_EXTRA_COMPONENT_DIRS
"/path/to/extra/component/" APPEND)

project(main)
```

See custom_bootloader/bootloader_extra_dir for an example of adding extra components to the bootloader build.

Configuration-Only Components

Special components which contain no source files, only `Kconfig.projbuild` and `Kconfig`, can have a one-line `CMakeLists.txt` file which calls the function `idf_component_register()` with no arguments specified. This function will include the component in the project build, but no library will be built *and* no header files will be added to any included paths.

Debugging CMake

For full details about CMake and CMake commands, see the CMake v3.22 documentation.

Some tips for debugging the ESP-IDF CMake-based build system:

- When CMake runs, it prints quite a lot of diagnostic information including lists of components and component paths.
- Running `cmake -DDEBUG=1` will produce more verbose diagnostic output from the IDF build system.
- Running `cmake` with the `--trace` or `--trace-expand` options will give a lot of information about control flow. See the cmake command line documentation.

When included from a project CMakeLists file, the `project.cmake` file defines some utility modules and global variables and then sets `IDF_PATH` if it was not set in the system environment.

It also defines an overridden custom version of the built-in CMake `project` function. This function is overridden to add all of the ESP-IDF specific project functionality.

Warning On Undefined Variables

By default, the function of warnings on undefined variables is disabled.

To enable this function, we can pass the `--warn-uninitialized` flag to CMake or pass the `--cmake-warn-uninitialized` flag to `idf.py` so it will print a warning if an undefined variable is referenced in the build. This can be very useful to find buggy CMake files.

Browse the /tools/cmake/project.cmake file and supporting functions in /tools/cmake/ for more details.

Example Component CMakeLists

Because the build environment tries to set reasonable defaults that will work most of the time, component `CMakeLists.txt` can be very small or even empty (see Minimal Component CMakeLists). However, overriding preset_component_variables is usually required for some functionality.

Here are some more advanced examples of component CMakeLists files.

Adding Conditional Configuration

The configuration system can be used to conditionally compile some files depending on the options selected in the project configuration.

`Kconfig`:

```
config FOO_ENABLE_BAR
    bool "Enable the BAR feature."
    help
        This enables the BAR feature of the FOO component.
```

`CMakeLists.txt`:

```
 set(srcs "foo.c" "more_foo.c")

 if(CONFIG_FOO_ENABLE_BAR)
     list(APPEND srcs "bar.c")
 endif()

idf_component_register(SRCS "${srcs}"
                       ...)
```

This example makes use of the CMake if function and list APPEND function.

This can also be used to select or stub out an implementation, as such:

`Kconfig`:

```
config ENABLE_LCD_OUTPUT
    bool "Enable LCD output."
    help
        Select this if your board has an LCD.

config ENABLE_LCD_CONSOLE
    bool "Output console text to LCD"
    depends on ENABLE_LCD_OUTPUT
    help
        Select this to output debugging output to the LCD

config ENABLE_LCD_PLOT
    bool "Output temperature plots to LCD"
    depends on ENABLE_LCD_OUTPUT
    help
        Select this to output temperature plots
```

`CMakeLists.txt`:

```
if(CONFIG_ENABLE_LCD_OUTPUT)
   set(srcs lcd-real.c lcd-spi.c)
else()
   set(srcs lcd-dummy.c)
endif()

# We need font if either console or plot is enabled
if(CONFIG_ENABLE_LCD_CONSOLE OR CONFIG_ENABLE_LCD_PLOT)
   list(APPEND srcs "font.c")
endif()

idf_component_register(SRCS "${srcs}"
                       ...)
```

Conditions Which Depend on the Target

The current target is available to CMake files via `IDF_TARGET` variable.

In addition to that, if target `xyz` is used (`IDF_TARGET=xyz`), then Kconfig variable `CONFIG_IDF_TARGET_XYZ` will be set.

Note that component dependencies may depend on `IDF_TARGET` variable, but not on Kconfig variables. Also one can not use Kconfig variables in `include` statements in CMake files, but `IDF_TARGET` can be used in such context.

Source Code Generation

Some components will have a situation where a source file isn't supplied with the component itself but has to be generated from another file. Say our component has a header file that consists of the converted binary data of a BMP file, converted using a hypothetical tool called bmp2h. The header file is then included in as C source file called graphics_lib.c:

```
add_custom_command(OUTPUT logo.h
     COMMAND bmp2h -i ${COMPONENT_DIR}/logo.bmp -o log.h
     DEPENDS ${COMPONENT_DIR}/logo.bmp
     VERBATIM)

add_custom_target(logo DEPENDS logo.h)
add_dependencies(${COMPONENT_LIB} logo)

set_property(DIRECTORY "${COMPONENT_DIR}" APPEND PROPERTY
     ADDITIONAL_CLEAN_FILES logo.h)
```

This answer is adapted from the CMake FAQ entry, which contains some other examples that will also work with ESP-IDF builds.

In this example, logo.h will be generated in the current directory (the build directory) while logo.bmp comes with the component and resides under the component path. Because logo.h is a generated file, it should be cleaned when the project is cleaned. For this reason, it is added to the ADDITIONAL_CLEAN_FILES property.

> **Note**
>
> If generating files as part of the project CMakeLists.txt file, not a component CMakeLists.txt, then use build property `PROJECT_DIR` instead of `${COMPONENT_DIR}` and `${PROJECT_NAME}.elf` instead of `${COMPONENT_LIB}`.)

If a a source file from another component included `logo.h`, then `add_dependencies` would need to be called to add a dependency between the two components, to ensure that the component source files were always compiled in the correct order.

Embedding Binary Data

Sometimes you have a file with some binary or text data that you'd like to make available to your component, but you don't want to reformat the file as a C source.

You can specify argument `EMBED_FILES` in the component registration, giving space-delimited names of the files to embed:

```
idf_component_register(...
                       EMBED_FILES server_root_cert.der)
```

Or if the file is a string, you can use the variable `EMBED_TXTFILES`. This will embed the contents of the text file as a null-terminated string:

```
idf_component_register(...
                       EMBED_TXTFILES server_root_cert.pem)
```

The file's contents will be added to the .rodata section in flash, and are available via symbol names as follows:

```
extern const uint8_t server_root_cert_pem_start[]
asm("_binary_server_root_cert_pem_start");
extern const uint8_t server_root_cert_pem_end[]
asm("_binary_server_root_cert_pem_end");
```

The names are generated from the full name of the file, as given in `EMBED_FILES`. Characters /, ., etc. are replaced with underscores. The _binary prefix in the symbol name is added by objcopy and is the same for both text and binary files.

To embed a file into a project, rather than a component, you can call the function `target_add_binary_data` like this:

```
target_add_binary_data(myproject.elf "main/data.bin" TEXT)
```

Place this line after the `project()` line in your project CMakeLists.txt file. Replace `myproject.elf` with your project name. The final argument can be `TEXT` to embed a null-terminated string, or `BINARY` to embed the content as-is.

For an example of using this technique, see the "main" component of the file_serving example protocols/http_server/file_serving/main/CMakeLists.txt - two files are loaded at build time and linked into the firmware.

It is also possible to embed a generated file:

```
add_custom_command(OUTPUT my_processed_file.bin
                   COMMAND my_process_file_cmd my_unprocessed_file.bin)
target_add_binary_data(my_target "my_processed_file.bin" BINARY)
```

In the example above, `my_processed_file.bin` is generated from `my_unprocessed_file.bin` through some command `my_process_file_cmd`, then embedded into the target.

To specify a dependence on a target, use the `DEPENDS` argument:

```
add_custom_target(my_process COMMAND ...)
target_add_binary_data(my_target "my_embed_file.bin" BINARY DEPENDS my_process)
```

The `DEPENDS` argument to `target_add_binary_data` ensures that the target executes first.

Code and Data Placements

ESP-IDF has a feature called linker script generation that enables components to define where its code and data will be placed in memory through linker fragment files. These files are processed by the build system, and is used to augment the linker script used for linking app binary. See Linker Script Generation for a quick start guide as well as a detailed discussion of the mechanism.

Fully Overriding the Component Build Process

Obviously, there are cases where all these recipes are insufficient for a certain component, for example when the component is basically a wrapper around another third-party component not originally intended to be compiled under this build system. In that case, it's possible to forego the ESP-IDF build system entirely by using a CMake feature called ExternalProject. Example component CMakeLists:

```
# External build process for quirc, runs in source dir and
# produces libquirc.a
externalproject_add(quirc_build
    PREFIX ${COMPONENT_DIR}
    SOURCE_DIR ${COMPONENT_DIR}/quirc
    CONFIGURE_COMMAND ""
    BUILD_IN_SOURCE 1
    BUILD_COMMAND make CC=${CMAKE_C_COMPILER} libquirc.a
    INSTALL_COMMAND ""
    )

 # Add libquirc.a to the build process
 add_library(quirc STATIC IMPORTED GLOBAL)
 add_dependencies(quirc quirc_build)

 set_target_properties(quirc PROPERTIES IMPORTED_LOCATION
      ${COMPONENT_DIR}/quirc/libquirc.a)
 set_target_properties(quirc PROPERTIES INTERFACE_INCLUDE_DIRECTORIES
      ${COMPONENT_DIR}/quirc/lib)

 set_directory_properties( PROPERTIES ADDITIONAL_CLEAN_FILES
      "${COMPONENT_DIR}/quirc/libquirc.a")
```

(The above CMakeLists.txt can be used to create a component named `quirc` that builds the quirc project using its own Makefile.)

- `externalproject_add` defines an external build system.
  - `SOURCE_DIR`, `CONFIGURE_COMMAND`, `BUILD_COMMAND` and `INSTALL_COMMAND` should always be set. `CONFIGURE_COMMAND` can be set to an empty string if the build system has no "configure" step. `INSTALL_COMMAND` will generally be empty for ESP-IDF builds.
  - Setting `BUILD_IN_SOURCE` means the build directory is the same as the source directory. Otherwise, you can set `BUILD_DIR`.
  - Consult the ExternalProject documentation for more details about `externalproject_add()`
- The second set of commands adds a library target, which points to the "imported" library file built by the external system. Some properties need to be set in order to add include directories and tell CMake where this file is.
- Finally, the generated library is added to ADDITIONAL_CLEAN_FILES. This means `make clean` will delete this library. (Note that the other object files from the build won't be deleted.)

ExternalProject Dependencies and Clean Builds

CMake has some unusual behavior around external project builds:

- ADDITIONAL_CLEAN_FILES only works when "make" or "ninja" is used as the build system. If an IDE build system is used, it won't delete these files when cleaning.
- However, the ExternalProject configure & build commands will *always* be re-run after a clean is run.

- Therefore, there are two alternative recommended ways to configure the external build command:

    1. Have the external `BUILD_COMMAND` run a full clean compile of all sources. The build command will be run if any of the dependencies passed to `externalproject_add` with `DEPENDS` have changed, or if this is a clean build (ie any of `idf.py clean`, `ninja clean`, or `make clean` was run.)
    2. Have the external `BUILD_COMMAND` be an incremental build command. Pass the parameter `BUILD_ALWAYS 1` to `externalproject_add`. This means the external project will be built each time a build is run, regardless of dependencies. This is only recommended if the external project has correct incremental build behavior, and doesn't take too long to run.

The best of these approaches for building an external project will depend on the project itself, its build system, and whether you anticipate needing to frequently recompile the project.

Custom Sdkconfig Defaults

> **Note**
>
> For more detailed information about `sdkconfig.defaults` file, please visit sdkconfig.defaults file in Project Configuration section.

For example projects or other projects where you don't want to specify a full sdkconfig configuration, but you do want to override some key values from the ESP-IDF defaults, it is possible to create a file `sdkconfig.defaults` in the project directory. This file will be used when creating a new config from scratch, or when any new config value hasn't yet been set in the `sdkconfig` file.

To override the name of this file or to specify multiple files, set the `SDKCONFIG_DEFAULTS` environment variable or set `SDKCONFIG_DEFAULTS` in top-level `CMakeLists.txt`. File names that are not specified as full paths are resolved relative to current project's directory.

When specifying multiple files, use a semicolon as the list separator. Files listed first will be applied first. If a particular key is defined in multiple files, the definition in the latter file will override definitions from former files.

Some of the IDF examples include a `sdkconfig.ci` file. This is part of the continuous integration (CI) test framework and is ignored by the normal build process.

Target-dependent Sdkconfig Defaults

If and only if an `sdkconfig.defaults` file exists, the build system will also attempt to load defaults from an `sdkconfig.defaults.TARGET_NAME` file, where `TARGET_NAME` is the value of `IDF_TARGET`. For example, for `esp32` target, default settings will be taken from `sdkconfig.defaults` first, and then from `sdkconfig.defaults.esp32`. If there are no generic default settings, an empty `sdkconfig.defaults` still needs to be created if the build system should recognize any additional target-dependent `sdkconfig.defaults.TARGET_NAME` files.

If `SDKCONFIG_DEFAULTS` is used to override the name of defaults file/files, the name of target-specific defaults file will be derived from `SDKCONFIG_DEFAULTS` value/values using the rule above. When there are multiple files in `SDKCONFIG_DEFAULTS`, target-specific file will be applied right after the file bringing it in, before all latter files in `SDKCONFIG_DEFAULTS`

For the following example, suppose the build target is `esp32` and these files exist in the project directory:

```
sdkconfig.defaults
sdkconfig.defaults.esp32
sdkconfig_devkit1
```

Example 1: `SDKCONFIG_DEFAULTS="sdkconfig.defaults;sdkconfig_devkit1"`

The build system will apply the files in this order:

1. `sdkconfig.defaults`.
2. `sdkconfig.defaults.esp32` - build system will always try to load target specific variation of every file listed in `SDKCONFIG_DEFAULTS`.
3. `sdkconfig_devkit1`.
4. The build system will also attempt to load a file named `sdkconfig_devkit1.esp32`, but because there is no file with this name, no additional file will be loaded.

Example 2: `SDKCONFIG_DEFAULTS="sdkconfig_devkit1"`

The build system will apply the files in this order:

1. `sdkconfig_devkit1`.
2. The build system will also attempt to load a file named `sdkconfig_devkit1.esp32`, but because there is no file with this name, no additional file will be loaded.

Warning

In the second example, the standard `sdkconfig.defaults` (or its target specific variation) is not applied, because it was not explicitly included in `SDKCONFIG_DEFAULTS`.

You can find more detailed information on how the project configuration works in the Project Configuration Guide. In the Configuration Files Structure and Relationships, you can find lower-level information about the configuration files.

Flash Arguments

There are some scenarios that we want to flash the target board without IDF. For this case we want to save the built binaries, esptool and esptool write-flash arguments. It's simple to write a script to save binaries and esptool.

After running a project build, the build directory contains binary output files (`.bin` files) for the project and also the following flashing data files:

- `flash_project_args` contains arguments to flash the entire project (app, bootloader, partition table, PHY data if this is configured).
- `flash_app_args` contains arguments to flash only the app.
- `flash_bootloader_args` contains arguments to flash only the bootloader.

You can pass any of these flasher argument files to `esptool` as follows:

```
esptool --chip esp32 write-flash @build/flash_project_args
```

Alternatively, it is possible to manually copy the parameters from the argument file and pass them on the command line.

The build directory also contains a generated file `flasher_args.json` which contains project flash information, in JSON format. This file is used by `idf.py` and can also be used by other tools which need information about the project build.

Building the Bootloader

The bootloader is a special "subproject" inside /components/bootloader/subproject. It has its own project CMakeLists.txt file and builds separate .ELF and .BIN files to the main project. However, it shares its configuration and build directory with the main project.

The subproject is inserted as an external project from the top-level project, by the file /components/bootloader/project_include.cmake. The main build process runs CMake for the subproject, which includes discovering components (a subset of the main components) and

generating a bootloader-specific config (derived from the main `sdkconfig`).

Writing Pure CMake Components

The ESP-IDF build system "wraps" CMake with the concept of "components", and helper functions to automatically integrate these components into a project build.

However, underneath the concept of "components" is a full CMake build system. It is also possible to make a component which is pure CMake.

Here is an example minimal "pure CMake" component CMakeLists file for a component named `json`:

```
add_library(json STATIC
cJSON/cJSON.c
cJSON/cJSON_Utils.c)

target_include_directories(json PUBLIC cJSON)
```

- This is actually an equivalent declaration to the espressif/cjson managed component.
- This file is quite simple as there are not a lot of source files. For components with a large number of files, the globbing behavior of ESP-IDF's component logic can make the component CMakeLists style simpler.)
- Any time a component adds a library target with the component name, the ESP-IDF build system will automatically add this to the build, expose public include directories, etc. If a component wants to add a library target with a different name, dependencies will need to be added manually via CMake commands.

Using Third-Party CMake Projects with Components

CMake is used for a lot of open-source C and C++ projects — code that users can tap into for their applications. One of the benefits of having a CMake build system is the ability to import these third-party projects, sometimes even without modification! This allows for users to be able to get functionality that may not yet be provided by a component, or use another library for the same functionality.

Importing a library might look like this for a hypothetical library `foo` to be used in the `main` component:

```
# Register the component
idf_component_register(...)

# Set values of hypothetical variables that control the build of `foo`
set(FOO_BUILD_STATIC OFF)
set(FOO_BUILD_TESTS OFF)

# Create and import the library targets
add_subdirectory(foo)

# Publicly link `foo` to `main` component
target_link_libraries(main PUBLIC foo)
```

For an actual example, take a look at build_system/cmake/import_lib. Take note that what needs to be done in order to import the library may vary. It is recommended to read up on the library's documentation for instructions on how to import it from other projects. Studying the library's CMakeLists.txt and build structure can also be helpful.

It is also possible to wrap a third-party library to be used as a component in this manner. For example, the mbedtls component is a wrapper for Espressif's fork of mbedtls. See its component CMakeLists.txt .

The CMake variable `ESP_PLATFORM` is set to 1 whenever the ESP-IDF build system is being used. Tests such as `if (ESP_PLATFORM)` can be used in generic CMake code if special IDF-specific logic is required.

Using ESP-IDF Components from External Libraries

The above example assumes that the external library `foo` (or `tinyxml` in the case of the `import_lib` example) doesn't need to use any ESP-IDF APIs apart from common APIs such as libc, libstdc++, etc. If the external library needs to use APIs provided by other ESP-IDF components, this needs to be specified in the external CMakeLists.txt file by adding a dependency on the library target `idf::<componentname>`.

For example, in the `foo/CMakeLists.txt` file:

```
add_library(foo bar.c fizz.cpp buzz.cpp)

if(ESP_PLATFORM)
  # On ESP-IDF, bar.c needs to include esp_flash.h from the spi_flash component
  target_link_libraries(foo PRIVATE idf::spi_flash)
endif()
```

Using Prebuilt Libraries with Components

Another possibility is that you have a prebuilt static library (`.a` file), built by some other build process.

The ESP-IDF build system provides a utility function `add_prebuilt_library` for users to be able to easily import and use prebuilt libraries:

```
add_prebuilt_library(target_name lib_path [REQUIRES req1 req2 ...] [PRIV_REQUIRES req1 req2 ...])
```

where:

- `target_name` - name that can be used to reference the imported library, such as when linking to other targets
- `lib_path` - path to prebuilt library; may be an absolute or relative path to the component directory

Optional arguments `REQUIRES` and `PRIV_REQUIRES` specify dependency on other components. These have the same meaning as the arguments for `idf_component_register`.

Take note that the prebuilt library must have been compiled for the same target as the consuming project. Configuration relevant to the prebuilt library must also match. If not paid attention to, these two factors may contribute to subtle bugs in the app.

For an example, take a look at build_system/cmake/import_prebuilt.

Using ESP-IDF in Custom CMake Projects

ESP-IDF provides a template CMake project for easily creating an application. However, in some instances the user might already have an existing CMake project or may want to create a custom one. In these cases it is desirable to be able to consume IDF components as libraries to be linked to the user's targets (libraries/executables).

It is possible to do so by using the build system APIs provided by tools/cmake/idf.cmake. For example:

```
cmake_minimum_required(VERSION 3.22)
project(my_custom_app C)

# Include CMake file that provides ESP-IDF CMake build system APIs.
include($ENV{IDF_PATH}/tools/cmake/idf.cmake)

# Include ESP-IDF components in the build, may be thought as an equivalent of
# add_subdirectory() but with some additional processing and magic for ESP-IDF build
# specific build processes.
idf_build_process(esp32)

# Create the project executable and plainly link the esp_libc component to it using
# its alias, idf::esp_libc.
add_executable(${CMAKE_PROJECT_NAME}.elf main.c)
target_link_libraries(${CMAKE_PROJECT_NAME}.elf idf::esp_libc)

# Let the build system know what the project executable is to attach more targets,
dependencies, etc.
idf_build_executable(${CMAKE_PROJECT_NAME}.elf)
```

The example in build_system/cmake/idf_as_lib demonstrates the creation of an application equivalent to hello world application using a custom CMake project.

Note

The IDF build system can only set compiler flags for source files that it builds. When an external CMakeLists.txt file is used and PSRAM is enabled, remember to add `-mfix-esp32-psram-cache-issue` to the C compiler arguments. See CONFIG_SPIRAM_CACHE_WORKAROUND for details of this flag.

ESP-IDF CMake Build System API

Idf-build-commands

```
idf_build_get_property(var property [GENERATOR_EXPRESSION])
```

Retrieve a build property *property* and store it in *var* accessible from the current scope. Specifying *GENERATOR_EXPRESSION* will retrieve the generator expression string for that property, instead of the actual value, which can be used with CMake commands that support generator expressions.

```
idf_build_set_property(property val [APPEND])
```

Set a build property *property* with value *val*. Specifying *APPEND* will append the specified value to the current value of the property. If the property does not previously exist or it is currently empty, the specified value becomes the first element/member instead.

```
idf_build_component(component_dir [component_source])
```

Present a directory *component_dir* that contains a component to the build system. Relative paths are converted to absolute paths with respect to current directory.

An optional *component_source* argument can be specified to indicate the source of the component. (default: "project_components")

This argument determines the overriding priority for components with the same name. For detailed information, see Multiple Components with the Same Name.

This argument supports the following values (from highest to lowest priority):

- "project_components" - project components
- "project_extra_components" - components from `EXTRA_COMPONENT_DIRS`
- "project_managed_components" - custom project dependencies managed by the IDF Component Manager
- "idf_components" - ESP-IDF built-in components, typically under /components

For instance, if a component named "json" is present as both "idf_components", and "project_components", the component as "project_components" takes precedence over the one as "idf_components".

Warning

All calls to this command must be performed before *idf_build_process*. This command does not guarantee that the component will be processed during build (see the *COMPONENTS* argument description for *idf_build_process*).

```
idf_build_process(target
                  [PROJECT_DIR project_dir]
                  [PROJECT_VER project_ver]
                  [PROJECT_NAME project_name]
                  [SDKCONFIG sdkconfig]
                  [SDKCONFIG_DEFAULTS sdkconfig_defaults]
                  [BUILD_DIR build_dir]
                  [COMPONENTS component1 component2 ...])
```

Performs the bulk of the behind-the-scenes magic for including ESP-IDF components such as component configuration, libraries creation, dependency expansion and resolution. Among these functions, perhaps the most important from a user's perspective is the libraries creation by calling each component's `idf_component_register`. This command creates the libraries for each component, which are accessible using aliases in the form idf::*component_name*. These aliases can be used to link the components to the user's own targets, either libraries or executables.

The call requires the target chip to be specified with *target* argument. Optional arguments for the call include:

- PROJECT_DIR - directory of the project; defaults to CMAKE_SOURCE_DIR
- PROJECT_NAME - name of the project; defaults to CMAKE_PROJECT_NAME
- PROJECT_VER - version/revision of the project; defaults to "1"
- SDKCONFIG - output path of generated sdkconfig file; defaults to PROJECT_DIR/sdkconfig or CMAKE_SOURCE_DIR/sdkconfig depending if PROJECT_DIR is set
- SDKCONFIG_DEFAULTS - list of files containing default config to use in the build (list must contain full paths); defaults to empty. For each value *filename* in the list, the config from file *filename.target*, if it exists, is also loaded.
- BUILD_DIR - directory to place ESP-IDF build-related artifacts, such as generated binaries, text files, components; defaults to CMAKE_BINARY_DIR
- COMPONENTS - select components to process among the components known by the build system (added via *idf_build_component*). This argument is used to trim the build. Other components are automatically added if they are required in the dependency chain, i.e., the public and private requirements of the components in this list are automatically added, and in turn the public and private requirements of those requirements, so on and so forth. If not specified, all components known to the build system are processed.

```
idf_build_executable(executable)
```

Specify the executable *executable* for ESP-IDF build. This attaches additional targets such as dependencies related to flashing, generating additional binary files, etc. Should be called after `idf_build_process`.

```
idf_build_get_config(var config [GENERATOR_EXPRESSION])
```

Get the value of the specified config. Much like build properties, specifying *GENERATOR_EXPRESSION* will retrieve the generator expression string for that config, instead of the actual value, which can be used with CMake commands that support generator expressions. Actual config values are only known after call to `idf_build_process`, however.

```
idf_build_add_post_elf_dependency(elf_filename dep_target)
```

Register a dependency that must run after the ELF is linked (post-ELF) and before the binary image is generated. This is useful when a component needs to post-process the ELF in place prior to `elf2image` execution (for example, inserting metadata, stripping sections, or generating additional symbol files). The dependency target `dep_target` must be a valid CMake target. If your rule reads or modifies the ELF, declare the ELF file as a `DEPENDS` of your custom command.

> **Important**
>
> When creating post-ELF steps, ensure the build graph remains acyclic:
>
> - Do not make the ELF itself the output of your custom command. Produce a separate output (for example, `app.elf.post`, `app.elf.symbols`, or a simple marker file).
> - If you must modify the ELF in place, also produce an additional output file and update its timestamp to be newer than the ELF after modification (for example, using `cmake -E touch`). This ensures the output file has a newer timestamp than the modified ELF, so CMake considers the rule satisfied and won't re-run it on subsequent builds.
>
> Following these rules ensures the post-ELF hook runs in the intended order without triggering infinite rebuild loops.

Example:

```
# Create a custom command to process the ELF file after linking
idf_build_get_property(elf_target EXECUTABLE GENERATOR_EXPRESSION)
add_custom_command(
    OUTPUT "${CMAKE_BINARY_DIR}/${CMAKE_PROJECT_NAME}.stripped_marker"
    COMMAND ${CMAKE_OBJCOPY} --strip-debug
            "$<TARGET_FILE:$<GENEX_EVAL:${elf_target}>>"
    COMMAND ${CMAKE_COMMAND} -E touch
            "${CMAKE_BINARY_DIR}/${CMAKE_PROJECT_NAME}.stripped_marker"
    DEPENDS "$<TARGET_FILE:$<GENEX_EVAL:${elf_target}>>"
)

# Wrap it in a custom target
add_custom_target(strip_elf DEPENDS
    "${CMAKE_BINARY_DIR}/${CMAKE_PROJECT_NAME}.stripped_marker"
)

# Register it to run after the ELF is linked but before the BIN is generated
idf_build_add_post_elf_dependency("${CMAKE_PROJECT_NAME}.elf" strip_elf)
```

```
idf_build_get_post_elf_dependencies(elf_filename out_var)
```

Retrieve the list of post-ELF dependencies registered for the given ELF file and store it in `out_var`.

Idf-build-properties

These are properties that describe the build. Values of build properties can be retrieved by using the build command `idf_build_get_property`. For example, to get the Python interpreter used for the build:

```
idf_build_get_property(python PYTHON)
message(STATUS "The Python interpreter is: ${python}")
```

- BUILD_DIR - build directory; set from `idf_build_process` BUILD_DIR argument
- BUILD_COMPONENTS - list of components included in the build; set by `idf_build_process`
- BUILD_COMPONENT_ALIASES - list of library alias of components included in the build; set by `idf_build_process`
- C_COMPILE_OPTIONS - compile options applied to all components' C source files
- COMPILE_OPTIONS - compile options applied to all components' source files, regardless of it being C or C++
- COMPILE_DEFINITIONS - compile definitions applied to all component source files
- CXX_COMPILE_OPTIONS - compile options applied to all components' C++ source files
- DEPENDENCIES_LOCK - lock file path used in component manager. The default value is *dependencies.lock* under the project path.
- EXECUTABLE - project executable; set by call to `idf_build_executable`
- EXECUTABLE_NAME - name of project executable without extension; set by call to `idf_build_executable`
- EXECUTABLE_DIR - path containing the output executable
- IDF_COMPONENT_MANAGER - the component manager is enabled by default, but if this property is set to `0` it was disabled by the IDF_COMPONENT_MANAGER environment variable
- IDF_PATH - ESP-IDF path; set from IDF_PATH environment variable, if not, inferred from the location of `idf.cmake`
- IDF_TARGET - target chip for the build; set from the required target argument for `idf_build_process`
- IDF_VER - ESP-IDF version; set from either a version file or the Git revision of the IDF_PATH repository
- INCLUDE_DIRECTORIES - include directories for all component source files
- KCONFIGS - list of Kconfig files found in components in build; set by `idf_build_process`
- KCONFIG_PROJBUILDS - list of Kconfig.projbuild files found in components in build; set by `idf_build_process`

- MINIMAL_BUILD - perform a minimal build by including only the "common" components required by all other components, along with the components that are direct or transitive dependencies only of the `main` component. By default, this property is disabled (set to `OFF`), but it can be enabled by setting it to `ON`.
- PROJECT_NAME - name of the project; set from `idf_build_process` PROJECT_NAME argument
- PROJECT_DIR - directory of the project; set from `idf_build_process` PROJECT_DIR argument
- PROJECT_VER - version of the project; set from `idf_build_process` PROJECT_VER argument
- PYTHON - Python interpreter used for the build; set from PYTHON environment variable if available, if not "python" is used
- SDKCONFIG - full path to output config file; set from `idf_build_process` SDKCONFIG argument
- SDKCONFIG_DEFAULTS - list of files containing default config to use in the build; set from `idf_build_process` SDKCONFIG_DEFAULTS argument
- SDKCONFIG_HEADER - full path to C/C++ header file containing component configuration; set by `idf_build_process`
- SDKCONFIG_CMAKE - full path to CMake file containing component configuration; set by `idf_build_process`
- SDKCONFIG_JSON - full path to JSON file containing component configuration; set by `idf_build_process`
- SDKCONFIG_JSON_MENUS - full path to JSON file containing config menus; set by `idf_build_process`

Idf-component-commands

```
idf_component_get_property(var component property [GENERATOR_EXPRESSION])
```

Retrieve a specified *component*'s component property, *property* and store it in *var* accessible from the current scope. Specifying *GENERATOR_EXPRESSION* will retrieve the generator expression string for that property, instead of the actual value, which can be used with CMake commands that support generator expressions.

```
idf_component_set_property(component property val [APPEND])
```

Set a specified *component*'s component property, *property* with value *val*. Specifying *APPEND* will append the specified value to the current value of the property. If the property does not previously exist or it is currently empty, the specified value becomes the first element/member instead.

```
idf_component_register([[SRCS src1 src2 ...] | [[SRC_DIRS dir1 dir2 ...] [EXCLUDE_SRCS
src1 src2 ...]]
                       [INCLUDE_DIRS dir1 dir2 ...]
                       [PRIV_INCLUDE_DIRS dir1 dir2 ...]
                       [REQUIRES component1 component2 ...]
                       [PRIV_REQUIRES component1 component2 ...]
                       [LDFRAGMENTS ldfragment1 ldfragment2 ...]
                       [REQUIRED_IDF_TARGETS target1 target2 ...]
                       [EMBED_FILES file1 file2 ...]
                       [EMBED_TXTFILES file1 file2 ...]
                       [KCONFIG kconfig]
                       [KCONFIG_PROJBUILD kconfig_projbuild]
                       [WHOLE_ARCHIVE])
```

Register a component to the build system. Much like the `project()` CMake command, this should be called from the component's CMakeLists.txt directly (not through a function or macro) and is recommended to be called before any other command. Here are some guidelines on what commands can **not** be called before `idf_component_register`:

- commands that are not valid in CMake script mode
- custom commands defined in project_include.cmake
- build system API commands except `idf_build_get_property`; although consider whether the property may not have been set yet

Commands that set and operate on variables are generally okay to call before `idf_component_register`.

The arguments for `idf_component_register` include:

- SRCS - component source files used for creating a static library for the component; if not specified, component is a treated as a config-only component and an interface library is created instead.
- SRC_DIRS, EXCLUDE_SRCS - used to glob source files (.c, .cpp, .S) by specifying directories, instead of specifying source files manually via SRCS. Note that this is subject to the limitations of globbing in CMake. Source files specified in EXCLUDE_SRCS are removed from the globbed files.
- INCLUDE_DIRS - paths, relative to the component directory, which will be added to the include search path for all other components which require the current component
- PRIV_INCLUDE_DIRS - directory paths, must be relative to the component directory, which will be added to the include search path for this component's source files only
- REQUIRES - public component requirements for the component
- PRIV_REQUIRES - private component requirements for the component; ignored on config-only components
- LDFRAGMENTS - component linker fragment files
- REQUIRED_IDF_TARGETS - specify the only target the component supports
- KCONFIG - override the default Kconfig file
- KCONFIG_PROJBUILD - override the default Kconfig.projbuild file
- WHOLE_ARCHIVE - if specified, the component library is surrounded by `-Wl,--whole-archive`, `-Wl,--no-whole-archive` when linked. This has the same effect as setting `WHOLE_ARCHIVE` component property.

The following are used for embedding data into the component, and is considered as source files when determining if a component is config-only. This means that even if the component does not specify source files, a static library is still created internally for the component if it specifies either:

- EMBED_FILES - binary files to be embedded in the component
- EMBED_TXTFILES - text files to be embedded in the component

Idf-component-properties

These are properties that describe a component. Values of component properties can be retrieved by using the build command `idf_component_get_property`. For example, to get the directory of the `freertos` component:

```
idf_component_get_property(dir freertos COMPONENT_DIR)
message(STATUS "The 'freertos' component directory is: ${dir}")
```

- COMPONENT_ALIAS - alias for COMPONENT_LIB used for linking the component to external targets; set by `idf_build_component` and alias library itself is created by `idf_component_register`

- COMPONENT_DIR - component directory; set by `idf_build_component`
- COMPONENT_OVERRIDEN_DIR - contains the directory of the original component if this component overrides another component
- COMPONENT_LIB - name for created component static/interface library; set by `idf_build_component` and library itself is created by `idf_component_register`
- COMPONENT_NAME - name of the component; set by `idf_build_component` based on the component directory name
- COMPONENT_TYPE - type of the component, whether LIBRARY or CONFIG_ONLY. A component is of type LIBRARY if it specifies source files or embeds a file
- COMPONENT_SOURCE - source of the component, one of "idf_components", "project_managed_components", "project_components", "project_extra_components". This is used to determine the override precedence of components with the same name.
- EMBED_FILES - list of files to embed in component; set from `idf_component_register` EMBED_FILES argument
- EMBED_TXTFILES - list of text files to embed in component; set from `idf_component_register` EMBED_TXTFILES argument
- INCLUDE_DIRS - list of component include directories; set from `idf_component_register` INCLUDE_DIRS argument
- KCONFIG - component Kconfig file; set by `idf_build_component`
- KCONFIG_PROJBUILD - component Kconfig.projbuild; set by `idf_build_component`
- LDFRAGMENTS - list of component linker fragment files; set from `idf_component_register` LDFRAGMENTS argument
- MANAGED_PRIV_REQUIRES - list of private component dependencies added by the IDF component manager from dependencies in `idf_component.yml` manifest file
- MANAGED_REQUIRES - list of public component dependencies added by the IDF component manager from dependencies in `idf_component.yml` manifest file
- PRIV_INCLUDE_DIRS - list of component private include directories; set from `idf_component_register` PRIV_INCLUDE_DIRS on components of type LIBRARY
- PRIV_REQUIRES - list of private component dependencies; set from value of `idf_component_register` PRIV_REQUIRES argument and dependencies in `idf_component.yml` manifest file
- REQUIRED_IDF_TARGETS - list of targets the component supports; set from `idf_component_register` REQUIRED_IDF_TARGETS argument
- REQUIRES - list of public component dependencies; set from value of `idf_component_register` REQUIRES argument and dependencies in `idf_component.yml` manifest file
- SRCS - list of component source files; set from SRCS or SRC_DIRS/EXCLUDE_SRCS argument of `idf_component_register`
- WHOLE_ARCHIVE - if this property is set to `TRUE` (or any boolean "true" CMake value: 1, `ON`, `YES`, `Y`), the component library is surrounded by `-Wl,--whole-archive`, `-Wl,--no-whole-archive` when linked. This can be used to force the linker to include every object file into the executable, even if the object file doesn't resolve any references from the rest

of the application. This is commonly used when a component contains plugins or modules which rely on link-time registration. This property is `FALSE` by default. It can be set to `TRUE` from the component CMakeLists.txt file.

File Globbing & Incremental Builds

The preferred way to include source files in an ESP-IDF component is to list them manually via SRCS argument to `idf_component_register`:

```
idf_component_register(SRCS library/a.c library/b.c platform/platform.c
                       ...)
```

This preference reflects the CMake best practice of manually listing source files. This could, however, be inconvenient when there are lots of source files to add to the build. The ESP-IDF build system provides an alternative way for specifying source files using `SRC_DIRS`:

```
idf_component_register(SRC_DIRS library platform
                       ...)
```

This uses globbing behind the scenes to find source files in the specified directories. Be aware, however, that if a new source file is added and this method is used, then CMake won't know to automatically re-run and this file won't be added to the build.

The trade-off is acceptable when you're adding the file yourself, because you can trigger a clean build or run `idf.py reconfigure` to manually re-run CMake. However, the problem gets harder when you share your project with others who may check out a new version using a source control tool like Git...

For components which are part of ESP-IDF, we use a third party Git CMake integration module (/tools/cmake/third_party/GetGitRevisionDescription.cmake) which automatically re-runs CMake any time the repository commit changes. This means if you check out a new ESP-IDF version, CMake will automatically rerun.

For project components (not part of ESP-IDF), there are a few different options:

- If keeping your project file in Git, ESP-IDF will automatically track the Git revision and re-run CMake if the revision changes.
- If some components are kept in a third git repository (not the project repository or ESP-IDF repository), you can add a call to the `git_describe` function in a component CMakeLists file in order to automatically trigger re-runs of CMake when the Git revision changes.

- If not using Git, remember to manually run `idf.py reconfigure` whenever a source file may change.
- To avoid this problem entirely, use `SRCS` argument to `idf_component_register` to list all source files in project components.

The best option will depend on your particular project and its users.

Build System Metadata

For integration into IDEs and other build systems, when CMake runs the build process generates a number of metadata files in the `build/` directory. To regenerate these files, run `cmake` or `idf.py reconfigure` (or any other `idf.py` build command).

- `compile_commands.json` is a standard format JSON file which describes every source file which is compiled in the project. A CMake feature generates this file, and many IDEs know how to parse it.
- `project_description.json` contains some general information about the ESP-IDF project, configured paths, etc.
- `flasher_args.json` contains esptool arguments to flash the project's binary files. There are also `flash_*_args` files which can be used directly with esptool. See Flash arguments.
- `CMakeCache.txt` is the CMake cache file which contains other information about the CMake process, toolchain, etc.
- `config/sdkconfig.json` is a JSON-formatted version of the project configuration values.
- `config/kconfig_menus.json` is a JSON-formatted version of the menus shown in menuconfig, for use in external IDE UIs.

JSON Configuration Server

A tool called `kconfserver` is provided to allow IDEs to easily integrate with the configuration system logic. `kconfserver` is designed to run in the background and interact with a calling process by reading and writing JSON over process stdin & stdout.

You can run `kconfserver` from a project via `idf.py confserver` or `ninja kconfserver`, or a similar target triggered from a different build generator.

For more information about `kconfserver`, see the esp-idf-kconfig documentation.

Build System Internals

Build Scripts

The listfiles for the ESP-IDF build system reside in /tools/cmake. The modules which implement core build system functionality are as follows:

- build.cmake - Build related commands i.e., build initialization, retrieving/setting build properties, build processing.
- component.cmake - Component related commands i.e., adding components, retrieving/setting component properties, registering components.
- kconfig.cmake - Generation of configuration files (sdkconfig, sdkconfig.h, sdkconfig.cmake, etc.) from Kconfig files.
- ldgen.cmake - Generation of final linker script from linker fragment files.
- target.cmake - Setting build target and toolchain file.
- utilities.cmake - Miscellaneous helper commands.

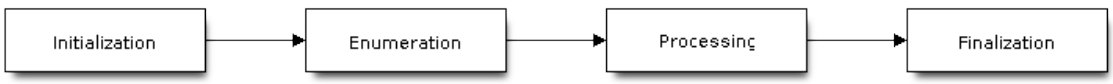
Aside from these files, there are two other important CMake scripts in /tools/cmake:

- idf.cmake - Sets up the build and includes the core modules listed above. Included in CMake projects in order to access ESP-IDF build system functionality.
- project.cmake - Includes `idf.cmake` and provides a custom `project()` command that takes care of all the heavy lifting of building an executable. Included in the top-level CMakeLists.txt of standard ESP-IDF projects.

The rest of the files in /tools/cmake are support or third-party scripts used in the build process.

Build Process

This section describes the standard ESP-IDF application build process. The build process can be broken down roughly into four phases:



ESP-IDF Build System Process

Initialization

This phase sets up necessary parameters for the build.

- **Upon inclusion of `idf.cmake` in `project.cmake`, the following steps are performed:**
    - Set `IDF_PATH` from environment variable or inferred from path to `project.cmake` included in the top-level CMakeLists.txt.
    - Add /tools/cmake to `CMAKE_MODULE_PATH` and include core modules plus the various helper/third-party scripts.
    - Set build tools/executables such as default Python interpreter.
    - Get ESP-IDF git revision and store as `IDF_VER`.
    - Set global build specifications i.e., compile options, compile definitions, include directories for all components in the build.
    - Add components in components to the build.
- **The initial part of the custom `project()` command performs the following steps:**
    - Set `IDF_TARGET` from environment variable or CMake cache and the corresponding `CMAKE_TOOLCHAIN_FILE` to be used.
    - Add components in `EXTRA_COMPONENT_DIRS` to the build.
    - Prepare arguments for calling command `idf_build_process()` from variables such as `COMPONENTS`/`EXCLUDE_COMPONENTS`, `SDKCONFIG`, `SDKCONFIG_DEFAULTS`.

The call to `idf_build_process()` command marks the end of this phase.

Enumeration

This phase builds a final list of components to be processed in the build, and is performed in the first half of `idf_build_process()`.

- Retrieve each component's public and private requirements. A child process is created which executes each component's CMakeLists.txt in script mode. The values of `idf_component_register` REQUIRES and PRIV_REQUIRES argument is returned to the parent build process. This is called early expansion. The variable `CMAKE_BUILD_EARLY_EXPANSION` is defined during this step.
- Recursively include components based on public and private requirements.
- Unless IDF Component Manager is disabled, it is called to resolve the dependencies of the components: - Looks for manifests and dependencies contained in the project. - Starts the version solving process to resolve the dependencies of the components. - When the version solving process succeeds, the IDF Component Manager downloads dependencies, integrates them into the build, and creates a `dependencies.lock` file that contains a list of the exact versions of the dependencies installed by the IDF Component Manager.

Processing

This phase processes the components in the build, and is the second half of `idf_build_process()`.

- Load project configuration from sdkconfig file and generate an sdkconfig.cmake and sdkconfig.h header. These define configuration variables/macros that are accessible from the build scripts and C/C++ source/header files, respectively.
- Include each component's `project_include.cmake`.
- Add each component as a subdirectory, processing its CMakeLists.txt. The component CMakeLists.txt calls the registration command, `idf_component_register` which adds source files, include directories, creates component library, links dependencies, etc.

Finalization

This phase is everything after `idf_build_process()`.

- Create executable and link the component libraries to it.
- Generate project metadata files such as project_description.json and display relevant information about the project built.

Browse /tools/cmake/project.cmake for more details.

Migrating from ESP-IDF GNU Make System

Some aspects of the CMake-based ESP-IDF build system are very similar to the older GNU Make-based system. The developer needs to provide values the include directories, source files etc. There is a syntactical difference, however, as the developer needs to pass these as arguments to the registration command, `idf_component_register`.

Automatic Conversion Tool

An automatic project conversion tool is available in *tools/cmake/convert_to_cmake.py* in ESP-IDF v4.x releases. The script was removed in v5.0 because of its *make* build system dependency.

No Longer Available in CMake

Some features are significantly different or removed in the CMake-based system. The following variables no longer exist in the CMake-based build system:

- `COMPONENT_BUILD_DIR`: Use `CMAKE_CURRENT_BINARY_DIR` instead.
- `COMPONENT_LIBRARY`: Defaulted to `$(COMPONENT_NAME).a`, but the library name could be overridden by the component. The name of the component library can no longer be overridden by the component.

- `CC`, `LD`, `AR`, `OBJCOPY`: Full paths to each tool from the gcc xtensa cross-toolchain. Use `CMAKE_C_COMPILER`, `CMAKE_C_LINK_EXECUTABLE`, `CMAKE_OBJCOPY`, etc instead. Full list here.
- `HOSTCC`, `HOSTLD`, `HOSTAR`: Full names of each tool from the host native toolchain. These are no longer provided, external projects should detect any required host toolchain manually.
- `COMPONENT_ADD_LDFLAGS`: Used to override linker flags. Use the CMake target_link_libraries command instead.
- `COMPONENT_ADD_LINKER_DEPS`: List of files that linking should depend on. target_link_libraries will usually infer these dependencies automatically. For linker scripts, use the provided custom CMake function `target_linker_scripts`.
- `COMPONENT_SUBMODULES`: No longer used, the build system will automatically enumerate all submodules in the ESP-IDF repository.
- `COMPONENT_EXTRA_INCLUDES`: Used to be an alternative to `COMPONENT_PRIV_INCLUDEDIRS` for absolute paths. Use `PRIV_INCLUDE_DIRS` argument to `idf_component_register` for all cases now (can be relative or absolute).
- `COMPONENT_OBJS`: Previously, component sources could be specified as a list of object files. Now they can be specified as a list of source files via `SRCS` argument to *idf_component_register*.
- `COMPONENT_OBJEXCLUDE`: Has been replaced with `EXCLUDE_SRCS` argument to `idf_component_register`. Specify source files (as absolute paths or relative to component directory), instead.
- `COMPONENT_EXTRA_CLEAN`: Set property `ADDITIONAL_CLEAN_FILES` instead but note CMake has some restrictions around this functionality.
- `COMPONENT_OWNBUILDTARGET` & `COMPONENT_OWNCLEANTARGET`: Use CMake ExternalProject instead. See Fully Overriding the Component Build Process for full details.
- `COMPONENT_CONFIG_ONLY`: Call `idf_component_register` without any arguments instead. See Configuration-Only Components.
- `CFLAGS`, `CPPFLAGS`, `CXXFLAGS`: Use equivalent CMake commands instead. See Controlling Component Compilation.

No Default Values

Unlike in the legacy Make-based build system, the following have no default values:

- Source directories (`COMPONENT_SRCDIRS` variable in Make, `SRC_DIRS` argument to `idf_component_register` in CMake)
- Include directories (`COMPONENT_ADD_INCLUDEDIRS` variable in Make, `INCLUDE_DIRS` argument to `idf_component_register` in CMake)

No Longer Necessary

- In the legacy Make-based build system, it is required to also set `COMPONENT_SRCDIRS` if `COMPONENT_SRCS` is set. In CMake, the equivalent is not necessary i.e., specifying `SRC_DIRS` to `idf_component_register` if `SRCS` is also specified (in fact, `SRCS` is ignored if `SRC_DIRS` is specified).

Flashing from Make

`make flash` and similar targets still work to build and flash. However, project `sdkconfig` no longer specifies serial port and baud rate. Environment variables can be used to override these. See Flashing with Ninja or Make for more details.

Application Examples

- build_system/wrappers demonstrates how to use a linker feature to redefine or override any public function in both ESP-IDF and the bootloader, allowing modification or extension of a function's default behavior.
- custom_bootloader/bootloader_override demonstrates how to override the second stage bootloader from a regular project, providing a custom bootloader that prints an extra message on startup, with the ability to conditionally override the bootloader based on certain conditions like target-dependency or Kconfig options.
- build_system/cmake/import_lib demonstrates how to import and use third-party libraries using ExternalProject CMake module.
- build_system/cmake/import_prebuilt demonstrates how to import a prebuilt static library into the ESP-IDF build system, build a component with dependencies, and link it to the main component, ultimately outputting the current running partition.
- build_system/cmake/idf_as_lib demonstrates the creation of an application equivalent to hello world application using a custom CMake project.
- build_system/cmake/multi_config demonstrates how to build multiple configurations of a single application from a single codebase, it is useful for creating binaries for multiple similar products.
- build_system/cmake/plugins demonstrates features of the ESP-IDF build system related to link time registration of plugins, allowing you to add multiple implementations of a certain feature without the need to make the application aware of all these implementations.

Was this page helpful?

